



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

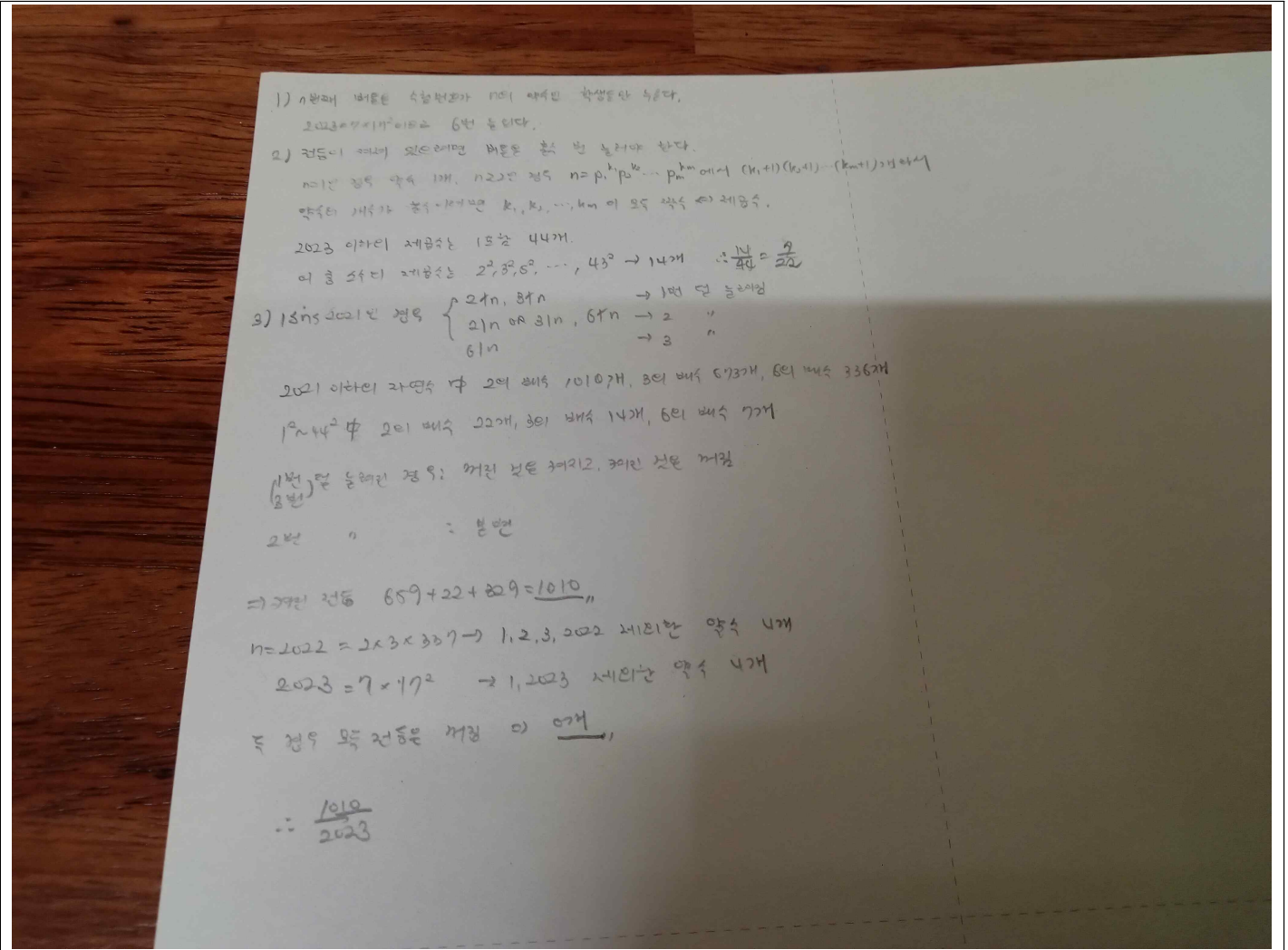
[제시문 1] 연세로에 2023 개의 전등이 1번부터 2023번까지 순서대로 놓여 있다. 모든 전등에는 버튼이 달려 있으며, 전등이 꺼져있을 때 버튼을 누르면 전등이 켜지고, 전등이 켜져있을 때 누르면 꺼진다. 수험번호가 k 인 학생은 연세로를 지나가며 전등 번호가 k 의 배수인 모든 전등의 버튼을 한 번씩 누른다고 하자. 다음 물음에 답하시오.

[문제 1-1] 수험번호가 1부터 2023까지의 총 2023명의 학생이 연세로를 지나갔다. 이때 2023 번째 전등의 버튼은 모두 몇 번 눌러졌을까? **[3점]**

[문제 1-2] 모든 전등이 처음에 꺼져있는 상태에서 수험번호가 1부터 2023까지의 학생이 연세로를 지나갔다. 2023명의 학생이 모두 지나간 후 켜져있는 전등 중에서 임의의 전등을 하나를 골랐을 때, 그 전등의 버튼이 총 세 번 눌러졌을 확률을 구하시오. **[5점]**

[문제 1-3] 모든 전등이 처음에 꺼져있는 상태에서 수험번호가 4부터 2021까지의 학생이 연세로를 지나갔다. 2018명의 학생이 모두 지나간 후 2023개의 전등 중에서 임의로 하나를 골랐을 때, 그 전등이 켜져있을 확률을 구하시오. **[9점]**

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

단순히 답을 구하는 것은 종이에 식만 적으면 어렵지 않게 할 수 있지만, 문제는 그 과정에서 2023개의 전등을 다루는 전체 구조가 한눈에 들어오지 않는다고 느꼈다. 문제의 핵심은 전등 n 이 몇 번 눌리는가를 세는 것이고, 그 횟수가 곧 n 의 약수의 개수와 연결된다는 점인데, 이는 풀이에서는 간단한 문장으로 끝나지만, 실제로 어떤 전등이 어떤 버튼들에 의해 눌리는가를 시각적으로 보여줌으로써 전체 구조를 체감할 수 있을 것이라고 기대했다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

제시된 문제상황을 html로 구현해줘. 과장된 홍보 문구는 제외하고, 사용법과 간단한 문제의 설명이 포함되어야 해. (이 이전에 다른 문제로 시도해 본 적이 있었는데, 그때 AI 특유의 과장된 표현(ex. html 표제어가 "10개의 원, 6개 선택 - 겁침이 넓이를 깎아내리는 순간을 눈으로 본다")이 강하게 느껴져서 이를 미연에 방지하고자 과장된 홍보 문구를 제외하라는 표현을 넣게 되었다.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 전등 2023개를 시각화하고, 시행 위치 k 에 따라 전등 상태를 갱신하며, 오른쪽 패널에 계산 결과와 증명 구조를 보여 주는 기본 기능은 작동했다. 그러나 실제 제출용으로 사용하기에는 다음과 같은 문제가 있었다.

첫째, 화면 하단 조작 패널의 높이가 고정되어 있어 일부 필터 버튼과 설명 영역이 잘렸다. 둘째, 필터 버튼이 많아질수록 작은 화면이나 브라우저 배율이 높은 환경에서 버튼 일부가 보이지 않았다. 셋째, 필터 영역을 스크롤하거나 패널을 확장하는 기능이 없어 사용자가 숨겨진 항목을 확인하기 어려웠다. 넷째, 시뮬레이터의 디자인은 유지할 만했지만, 조작 패널의 접근성과 반응형 구조는 개선해야 했다.

이후 1차 수정 후에는 드래그 스크롤 기능이 버튼 클릭 이벤트를 가로채 재생 버튼과 필터 버튼이 눌리지 않는 문제가 추가로 발견되었다. 따라서 하단 패널 확장 기능뿐 아니라 버튼 클릭 이벤트와 드래그 이벤트가 서로 충돌하지 않도록 추가 수정이 필요했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

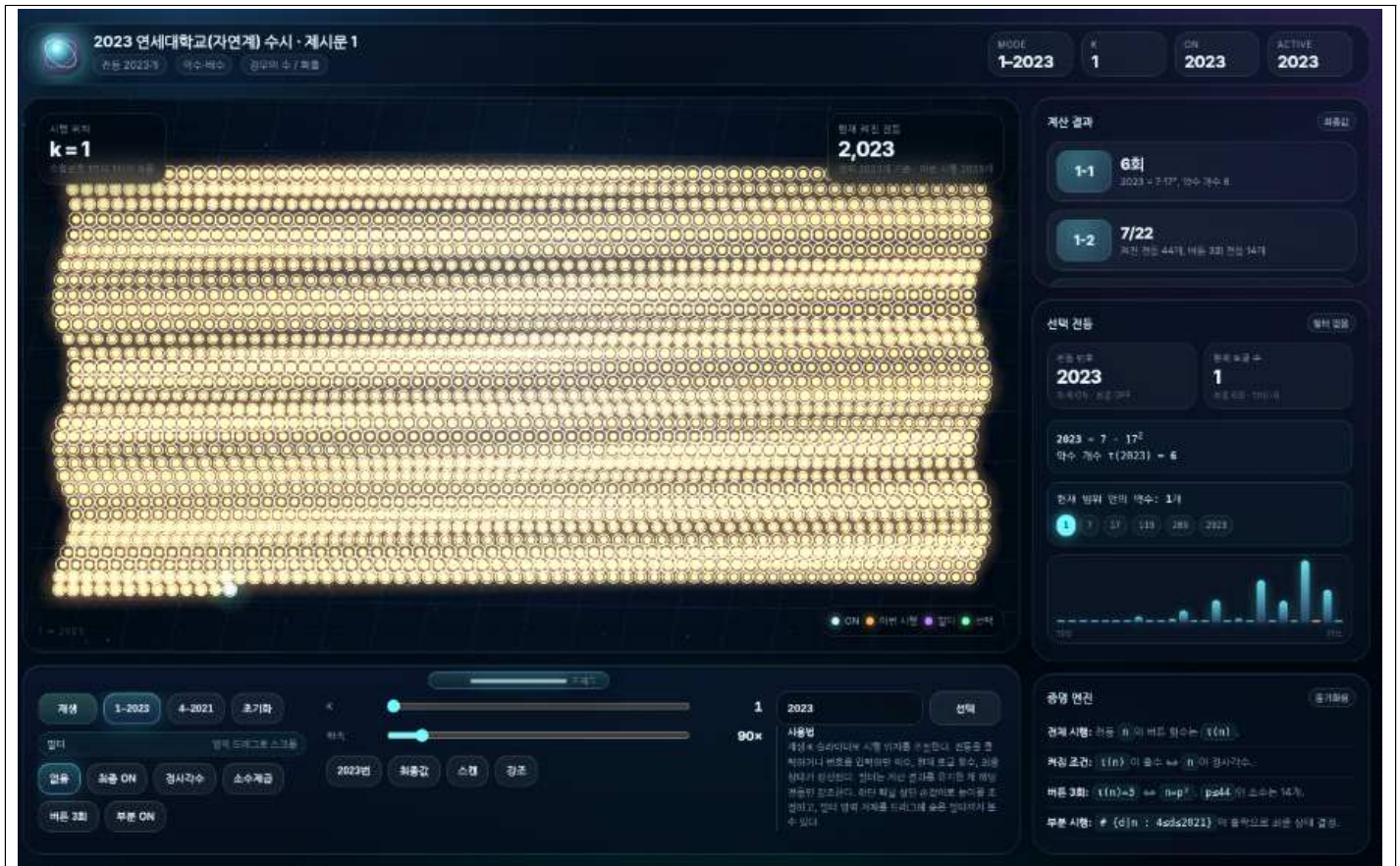
2번째 프롬프트: 밑에 내용이 잘려. 드래그 기능 만들어서 필터의 내용도 볼 수 있게 해줘.

이 프롬프트를 통해 하단 조작 패널에 드래그 손잡이를 추가하고, 패널 높이를 사용자가 직접 조절할 수 있도록 수정했다. 또한 필터 영역 자체를 드래그해서 숨겨진 필터 버튼까지 볼 수 있게 했다.

3번째 프롬프트: 디자인은 완벽해졌는데, 재생 버튼이랑 각종 필터 버튼들이 눌려지지 않아.

이 프롬프트를 통해 드래그 스크롤 기능과 버튼 클릭 기능이 충돌하지 않도록 이벤트 처리를 분리했다. 버튼 위에서 드래그 스크롤이 시작되지 않게 하고, 실제로 드래그가 발생한 경우에만 클릭 이벤트를 막도록 수정하여 재생, 정지, 필터, 초기화, 스캔, 강조 버튼이 정상적으로 작동하게 했다.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

시뮬레이터를 열면 가운데 큰 영역에 1번부터 2023번까지의 전등이 배열되어 있다. 각 전등은 현재 시행 위치 k 까지 버튼이 눌린 결과에 따라 켜짐 또는 꺼짐 상태로 표시된다. 오른쪽에는 계산 결과, 선택 전등 정보, 증명 엔진이 표시되고, 아래쪽에는 재생 버튼, 모드 버튼, 필터 버튼, 슬라이더, 전등 번호 입력창이 있다.

재생 버튼을 누르면 k 값이 자동으로 증가하면서 시행 과정이 진행된다. 다시 누르면 정지된다. 슬라이더를 움직이면 원하는 시행 위치로 바로 이동할 수 있다. 예를 들어 $k=100$ 이면 1번부터 100번까지의 버튼을 누른 상황이 표시된다. 배속 슬라이더를 움직이면 자동 재생 속도를 조절할 수 있다.

1-2023 버튼을 누르면 1번부터 2023번까지 전체 버튼을 시행하는 모드가 된다. 4-2021 버튼을 누르면 4번부터 2021번까지의 버튼만 시행하는 모드가 된다.

전등을 직접 클릭하거나 오른쪽 아래 입력창에 전등 번호를 입력한 뒤 선택 버튼을 누르면 해당 전등의 약수, 현재 토글 횟수, 최종 상태가 오른쪽 패널에 표시된다.

필터 버튼을 이용하면 특정 조건의 전등만 강조할 수 있다. 최종 ON은 최종적으로 켜진 전등을, 정사각수는 정사각수 번호의 전등을, 소수제곱은 p^2 꼴의 전등을, 버튼 3회는 정확히 3번 눌린 전등을, 부분 ON은 4-2021 모드에서 켜지는 전등을 강조한다.

2023번 버튼은 2023번 전등을 즉시 선택한다. 최종값 버튼은 시행 위치를 마지막까지 이동시킨다. 스캔 버튼은 전체 전등의 패턴을 훑어보는 효과를 준다. 강조 버튼은 현재 선택된 전등을 시각적으로 더 강하게 표시한다.

하단 패널의 위쪽 손잡이를 드래그하면 조작 패널의 높이를 조절할 수 있다. 필터 영역 자체를 좌우 또는 상하로 드래그하면 숨겨진 필터 버튼까지 볼 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

시뮬레이터는 의도한 대로 문제의 구조를 파악하는 데 도움이 되었다. 가장 큰 장점은 수식적으로 이해했던 전등 n 은 n 의 약수에 해당하는 버튼이 눌릴 때마다 한 번씩 상태가 바뀐다는 사실을 아무 생각 없이 시각적으로 직접 확인할 수 있다는 점이다. 예를 들어 2023번 전등을 선택하면 $2023 = 7 \times 17^2$ 이고 약수의 개수가 6개라는 정보가 바로 표시되어, 문제 1-1의 결과가 왜 6회인지 확인할 수 있다.

문제 1-2의 검토에도 도움이 되었다. 전체 시행 후 켜진 전등은 약수의 개수가 홀수인 전등, 즉 제곱수 번호의 전등이라는 점이 필터를 통해 드러난다. 2023 이하의 제곱수가 44개이므로 최종적으로 켜진 전등의 수가 44개라는 사실을 확인할 수 있고, 버튼이 정확히 3회 눌린 전등은 $\tau(n)=3$ 인 경우, 즉 $n=p^2$ 꼴이므로 $p \leq 44$ 인 소수의 개수 14개와 연결된다. 따라서 확률 $7/22$ 를 시각적으로 검토할 수 있었다.

문제 1-3에서도 도움이 되었다. 전체 시행과 4-2021 부분 시행을 전환하면 시행 범위가 바뀌었을 때 전등의 최종 상태가 어떻게 달라지는지 바로 확인할 수 있다. 이 과정에서 양끝의 작은 약수나 큰 약수가 제외될 때 홀짝성이 변한다는 점을 관찰할 수 있었고, 최종 결과 1010/2023을 단순 계산 결과가 아니라 패턴의 변화로 이해할 수 있었다.

다만 시뮬레이터가 모든 풀이를 대신해 주는 것은 아니다. 정답을 얻는 데에는 도움이 되지만, 논술 답안에서는 왜 약수의 개수와 전등의 토글 횟수가 같은지, 왜 약수의 개수가 홀수인 수가 제곱수인지, 왜 $\tau(n)=3$ 인 경우, 즉 $n=p^2$ 인지 등에 대한 기본적인 수학적 지식이 필요하다. 따라서 시뮬레이터는 풀이를 발견하고 검토하는 도구로는 매우 유용하지만, 인간의 이해 역시 중요하다.

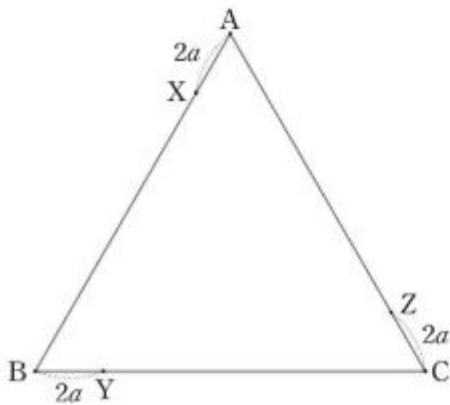
[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

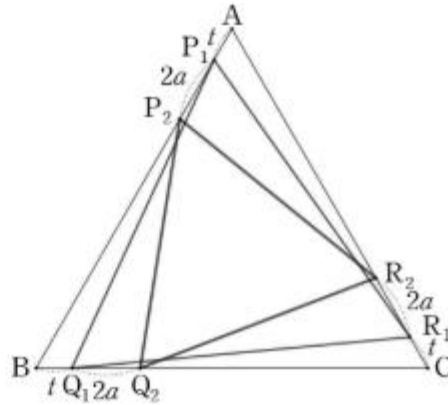
문제 1.

[그림 1]과 같이 한 변의 길이가 2인 정삼각형 ABC가 있다. 세 점 X, Y, Z는 각각 변 AB, 변 BC, 변 CA 위의 점으로 $\overline{AX} = \overline{BY} = \overline{CZ} = 2a$ 를 만족한다.

(단, a 는 $0 < a < 1$ 인 실수)



[그림 1] 삼각형 ABC와 점 X, Y, Z



[그림 2] 시각 t 에 점들의 위치

정삼각형 ABC의 세 변 위를 여섯 점 $P_1, P_2, Q_1, Q_2, R_1, R_2$ 가 다음 [규칙]에 따라 [그림 2]와 같이 움직인다.

[규칙]

- (가) 두 점 P_1, P_2 는 각각 점 A, X에서 시각 $t=0$ 에 동시에 출발하여 변 AB를 따라 속도 1로 점 B를 향해 움직인다.
- (나) 두 점 Q_1, Q_2 는 각각 점 B, Y에서 시각 $t=0$ 에 동시에 출발하여 변 BC를 따라 속도 1로 점 C를 향해 움직인다.
- (다) 두 점 R_1, R_2 는 각각 점 C, Z에서 시각 $t=0$ 에 동시에 출발하여 변 CA를 따라 속도 1로 점 A를 향해 움직인다.
- (라) 시각 $t=2-2a$ 가 되어 세 점 P_2, R_2, Q_2 가 각각 점 B, C, A에 도착하면, 여섯 점 $P_1, P_2, Q_1, Q_2, R_1, R_2$ 는 모두 이동을 멈춘다.

- 1-1. 시각 $t (0 \leq t \leq 2-2a)$ 에서의 삼각형 $P_1Q_1R_1$ 의 넓이를 t 에 대한 식으로 나타내시오.
- 1-2. 두 삼각형 $P_1Q_1R_1$ 과 $P_2Q_2R_2$ 의 넓이가 같아지는 시각 t_0 을 a 에 대한 식으로 나타내시오.
- 1-3. 시각 $t (0 \leq t \leq 2-2a)$ 에서의 삼각형 $P_1Q_1R_1$ 의 넓이와 삼각형 $P_2Q_2R_2$ 의 넓이의 곱을 $f(t)$ 라 하자. 닫힌구간 $[0, 2-2a]$ 에서 함수 $f(t)$ 가 $t=t_0$ 에서 최솟값을 가지도록 하는 실수 a 의 값의 범위를 구하시오. (단, t_0 은 문제 1-2에서 구한 값이다.)
- 1-4. 닫힌구간 $[0, 2-2a]$ 에서 함수 $f(t)$ 가 $t=t_0$ 에서 최댓값을 가지도록 하는 실수 a 의 값의 범위를 구하시오. (단, t_0 은 문제 1-2에서 구한 값이다.)

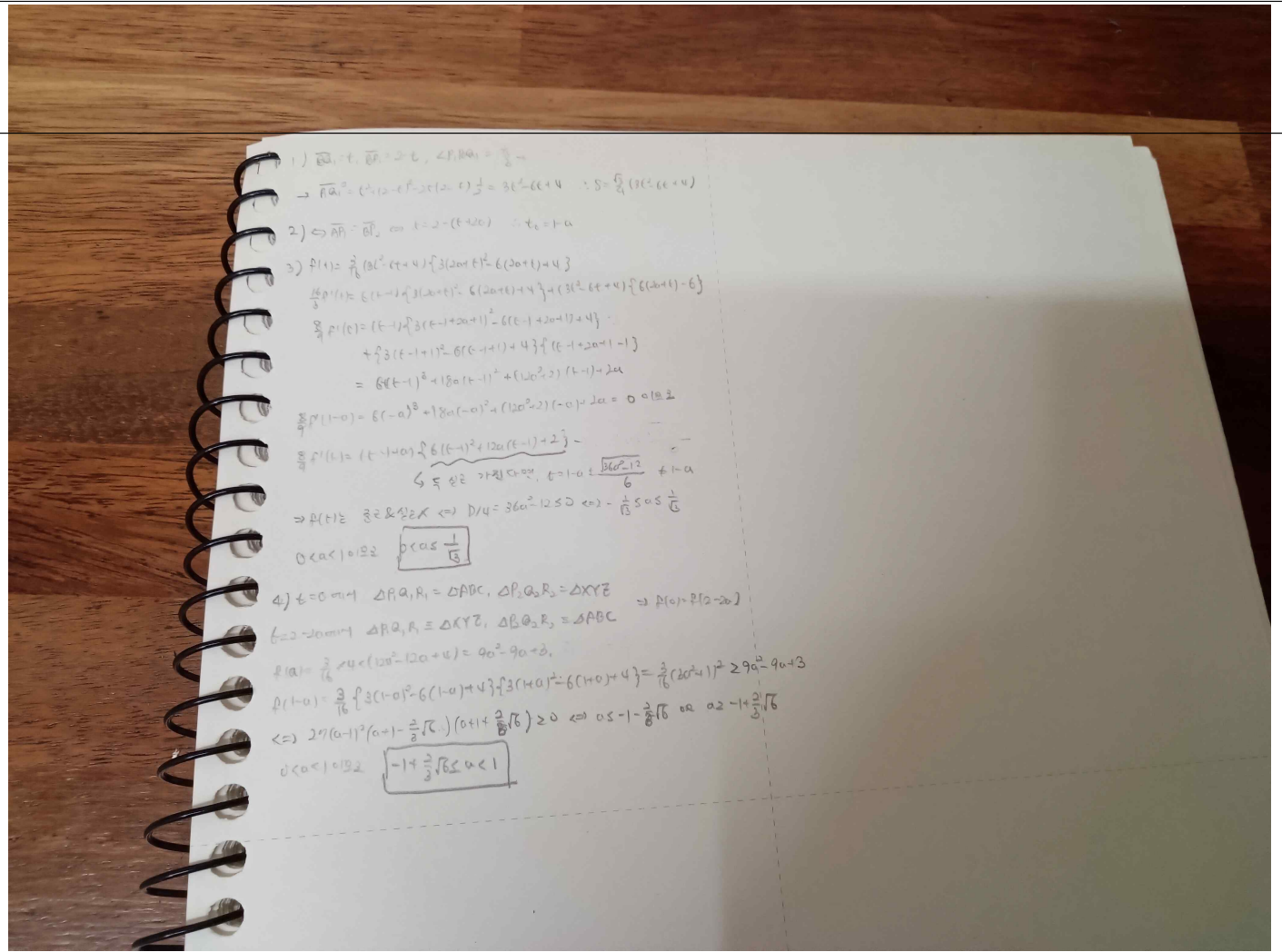
2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제에서는 정삼각형의 세 변 위에서 여섯 개의 점이 동시에 움직이고, 그 점들이 만드는 두 삼각형의 넓이 $A_1(t), A_2(t), F(t) = A_1(t)A_2(t)$ 의 극값을 분석하는 문제이다. 문제를 풀어나가기 위해 t 가 변할 때 점들이 어디에 있는지, 두 삼각형이 어떻게 커지고 작아지는지, 두 넓이의 곱이 왜 특정 시점에서 극값을 갖는지 등에 대한 부분이 한눈에 보이지 않아서 처음 문제를 푸는 과정에서 어려움이 많았다. 따라서 이 문제에서의 변화의 구조를 파악하기 위해 시뮬레이션이 큰 도움이 될 수 있으리라 생각했다.

4. 사용한 AI 플랫폼을 모두 적으세요.

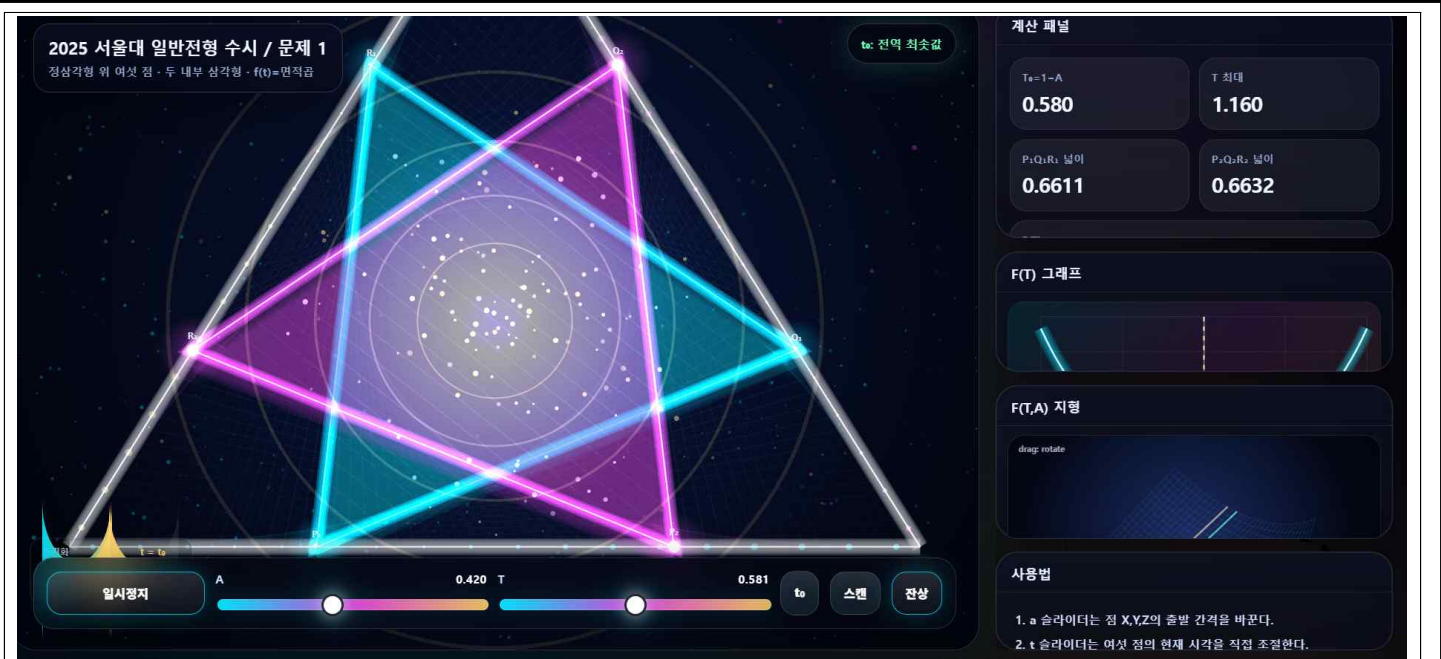
ChatGPT



5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

제시된 문제상황을 html로 구현해줘. 과장된 홍보 문구는 제외하고, 사용법과 간단한 문제의 설명이 포함되어야 해.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터에서 수정해야 할 사항은 크게 세 가지였다.

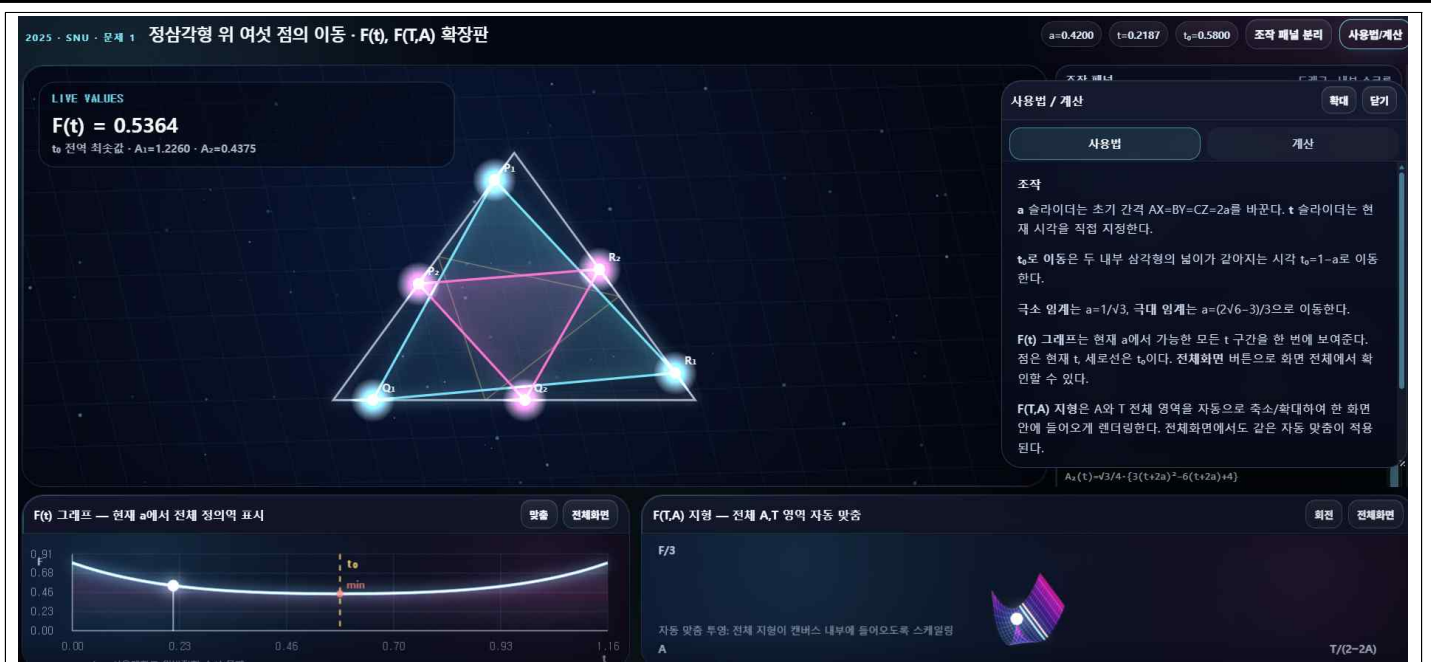
첫째, 처음 시뮬레이터는 시각 효과는 있었지만, 문제의 수학적 구조가 한눈에 드러나지 않았다. 점들이 움직이는 장면은 보였지만, $F(t)$ 그래프가 충분히 강조되지 않았고, a 값이 변할 때 그래프 전체가 어떻게 바뀌는지 파악하기 어려웠다. 따라서 단순히 움직이는 도형을 보여주는 수준을 넘어서, $F(t)$ 그래프와 $F(t,a)$ 지형을 함께 보여줄 필요가 있었다.

둘째, 작은 화면에서 조작 패널과 계산 패널의 일부가 잘리는 문제가 있었다. 슬라이더와 수치 카드가 화면 아래쪽으로 밀려 내려가면서 $F(t)$, $F'(t)$, 계산식, 사용법 등을 모두 읽기 어려웠다. 시뮬레이터는 누구나 같은 화면에서 내용을 확인할 수 있어야 하므로, 패널 내부 스크롤과 드래그 기능이 필요했다.

셋째, $F(t)$ 그래프와 $F(t,a)$ 지형이 전체적으로 보이지 않는 문제가 있었다. 그래프나 지형의 일부만 보이면 극대·극소의 전체 구조를 비교할 수 없기 때문에, 기본 상태에서 전체 모양이 보이도록 자동 맞춤이 필요했다. 또한 필요할 때 그래프와 지형을 화면 전체로 확대해서 볼 수 있는 기능도 추가해야 했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

이 시뮬레이터는 정삼각형 위를 움직이는 여섯 점과, 그 점들이 만드는 두 삼각형의 넓이 및 넓이의 곱을 관찰하기 위한 도구이다. 사용 방법은 다음과 같다.

먼저 HTML 파일을 실행하면 화면에 정삼각형과 여섯 개의 이동점이 나타난다. P_1, Q_1, R_1 은 첫 번째 삼각형을 만들고, P_2, Q_2, R_2 는 두 번째 삼각형을 만든다. 두 삼각형의 넓이는 각각 $A_1(t), A_2(t)$ 로 표시되며, 두 넓이의 곱은 $F(t)$ 로 표시된다.

오른쪽 또는 상단의 조작 패널에서 a 슬라이더를 움직이면 초기 간격 $2a$ 가 바뀐다. a 값이 바뀌면 두 번째 삼각형의 시작 위치가 달라지고, 이에 따라 $F(t)$ 그래프의 전체 모양도 달라진다.

t 슬라이더를 움직이면 현재 시각 t 를 직접 조절할 수 있다. 슬라이더를 움직일 때마다 점들의 위치, 두 삼각형의 넓이, $F(t), F'(t)$ 값이 실시간으로 바뀐다. 이를 통해 특정 순간에 두 삼각형의 넓이가 어떻게 변하는지 확인할 수 있다.

재생/정지 버튼을 누르면 점들의 이동을 자동으로 재생하거나 멈출 수 있다. 자동 재생 상태에서는 t 값이 계속 증가하면서 점들이 정삼각형의 변을 따라 움직인다. 정지 상태에서는 사용자가 직접 t 값을 조절하며 특정 순간을 분석할 수 있다.

t_0 로 이동 버튼을 누르면 $t_0 = 1 - a$ 인 시각으로 이동한다. 이 시각은 두 내부 삼각형의 넓이 관계가 중요한 지점이며, $F(t)$ 의 극값 판정과 관련이 있다. 이 버튼을 사용하면 문제 풀이에서 중요한 순간을 바로 확인할 수 있다.

극소 임계, 극대 임계 버튼은 a 값을 특정 임계값으로 이동시킨다. 극소 임계는 $a=1/\sqrt{3}$, 극대 임계는 $a=(2\sqrt{6}-3)/3$ 에 해당한다. 이 버튼을 이용하면 a 값에 따라 t_0 의 성격이 어떻게 달라지는지 비교할 수 있다.

화면의 $F(t)$ 그래프 패널에서는 현재 a 값에서 가능한 전체 t 구간의 $F(t)$ 그래프를 볼 수 있다. 그래프에는 현재 t 의 위치와 t_0 위치가 함께 표시된다. 그래프가 작게 보이면 전체화면 버튼을 눌러 크게 볼 수 있다.

$F(t,a)$ 지형 패널에서는 a 와 t 가 함께 변할 때 F 값이 어떻게 달라지는지 3차원 지형처럼 볼 수 있다. 이 지형은 a 가 바뀔 때 그래프의 모양이 어떻게 변하는지를 한 번에 보여준다. 필요하면 전체화면으로 확대해서 볼 수 있고, 회전 기능을 켜서 지형의 입체적 구조를 관찰할 수 있다.

사용법/계산 패널은 드래그해서 위치를 옮길 수 있고, 내부 스크롤을 통해 모든 내용을 읽을 수 있다. 계산 탭에서는 사용된 공식과 도함수, 극값 판정 조건을 확인할 수 있다. 화면이 작아서 일부 내용이 보이지 않을 경우 패널 내부를 스크롤하거나 패널을 이동시키면 된다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

의도한 대로 문제의 구조를 파악하고, 점들의 이동을 직접 보는 데에 큰 도움이 되었다. 특히 두 삼각형이 동시에 움직이는 이와 같은 문제는 종이 위에 정지된 그림을 그리는 것만으로는 이해하기 쉽지 않았는데, 시뮬레이터에서는 동적 그림을 표현할 수 있어 문제 상황이 자연스럽게 이해된다. 뿐만 아니라, $A_1(t), A_2(t), F(t), F'(t)$ 도 실시간으로 표시되어 그래프를 도형의 변화와 함수값의 변화와 동시에 연결시켜 이해할 수 있다. 예를 들어, $t_0 = 1 - a$ 로 이동했을 때 $F'(t)$ 가 어떻게 변하고 $F(t)$ 그래프에서 어떤 위치에 해당하는지 바로 확인할 수 있었다. 게다가 $F(t, a)$ 를 3차원 구조로 표현해서 여러 a 값에 대한 전체 변화까지 보여주는데, 이를 통해 a 의 범위에 따라 경우를 나누어서 풀어야겠다는 힌트도 얻을 수 있을 것이다.

다만, 시뮬레이터는 직관적으로 a 의 값에 따른 그래프의 개형을 대략적으로 파악하는 데는 도움을 주지만, 정확한 a 의 값이 왜 그렇게 나오는지 이해하는 데에는 도움이 되지 않는다. 이는 우리가 식을 써서 논리적으로 증명해야 하는 부분이다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

생성형 AI가 가장 도움이 된 부분은 시뮬레이터를 실제로 구현하는 과정이었다. 내가 문제의 핵심 아이디어를 설명하면, AI는 HTML, CSS, JavaScript를 이용해서 점의 이동, 그래프, 넓이 변화, 3차원 지형 같은 시각 자료를 빠르게 만들어 주었다. 1번 문제에서는 여러 조합을 시각적으로 확인하게 해 주었고, 2번 문제에서는 $a, t, F(t)$ 의 관계를 그래프와 지형으로 보여 주는 데 도움이 되었다. 두 문제 모두 조합, 도형, 넓이, 함수의 극대·극소 같은 개념이 섞여 있었기 때문에 시각화의 효과가 컸다.

하지만 AI가 만든 결과를 그대로 믿는 것은 위험하다고 느꼈다. 실제로 어떤 변수를 잡아야 하는지, 넓이 식이 맞는지, 극값을 어떻게 판정해야 하는지 판단하는 부분은 내가 직접 해야 했다. 예를 들어 서울대 문제에서 $t_0 = 1 - a$ 가 왜 중요한지, $F(t) = A_1(t)A_2(t)$ 로 놓는 것이 적절한지, 도함수의 부호를 어떻게 해석해야 하는지는 단순히 화려한 화면과 수학적 직관으로 해결되는 부분이 아니었다.

정리하면, 생성형 AI는 구현, 시각화, 반복 수정, 화면 구성에서 큰 도움을 줄 수 있었다. 반면 인간이 주도해야 하는 부분은 문제의 핵심 조건 파악, 변수 설정, 식의 타당성 검토, 극값 판단, 최종 풀이 논리 정리라고 생각한다. AI는 도구이고, 수학적 판단은 결국 학생 본인이 해야 한다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이번 활동을 통해 복잡한 문제를 해결할 때 꼭 처음부터 완벽한 답을 구하려고 하기보다, 먼저 모델을 만들고 시각적으로 확인하는 방식이 유용하다는 것을 느꼈다. 내 진로 분야인 화학에서도 많은 분자들의 상호작용을 양자역학적으로 설명하기 위해서는 대단히 많은 계산량이 필요하고, 그렇기에 많은 근사를 사용하고 있다. 이 근사 모델을 만들기 위해 이전까지는 소수의 천재들이 제안하는 직관에 의존해야 했지만, 이제는 AI를 이용해 간단한 시뮬레이터나 시각화 도구를 만들면 문제의 구조를 더 빨리 파악할 수 있을 것이라고 생각했다.

또한 AI와 협업할 때는 “알아서 잘 만들어 줘”라고 하는 것보다, 내가 원하는 기능을 구체적으로 말해야 결과가 좋아진다는 점도 알게 되었다. 처음에는 보기 좋은 시뮬레이터를 요구했지만, 실제로는 그래프가 잘리거나 패널이 안 보이는 문제가 있었다. 이후에 “그래프 전체가 보이게 해 달라”, “패널을 드래그하거나 스크롤할 수 있게 해 달라” 처럼 구체적으로 요청하자 결과물이 더 실용적으로 개선되었다.

이제 AI가 나보다 수학 문제는 분명히 잘 풀게 되었지만, 여전히 어느 정도의 규모가 있는 프로젝트에서는 오류가 빈번하게 나타나는 한다. 이 경험은 향후 실무에서 AI를 사용할 때 알아서 하라고 맡기기보다는, 초안 제작 → 검토 → 수정 요청 → 재검토의 과정으로 인간의 개입이 있어야 보다 검증된 자료가 도출될 수 있다는 시사점을 준다. 즉, AI를 잘 쓰려면 문제를 맡기는 능력보다 문제를 정의하고 결과를 검증하는 능력이 더 중요하다고 생각했다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

보고서를 작성하면서 시뮬레이터를 만드는 것보다, 내가 왜 그렇게 만들었는지 설명하는 것이 생각보다 더 어렵다고 느꼈다. 처음에는 화면이 잘 나오면 끝이라고 생각했는데, 보고서에는 문제 선정 이유, 수정 과정, 사용 방법, 도움이 된 점까지 정리해야 해서 내가 한 활동을 다시 되짚어 보게 되었다.

특히 AI가 만든 결과물을 그대로 제출하는 것이 아니라, 어떤 부분을 수정했고 왜 수정했는지를 쓰면서 내가 실제로 개입한 부분이 무엇인지 확인할 수 있었다. 시뮬레이터가 완벽한 풀이를 대신해 주는 것은 아니지만, 복잡한 수학 문제를 이해하는 데 도움을 줄 수 있다는 점은 분명히 느꼈다.